

SPECTER

// TECHNICAL DOCUMENTATION

BASE NETWORK INTELLIGENCE // AUTONOMOUS AGENT

VERSION 1.0 // 2026

CHAIN: BASE NETWORK (8453)

REPOSITORY: github.com/ariam2025/specter

SPECTER is an open-source autonomous intelligence agent deployed on Base Network. It runs continuously via GitHub Actions, scanning for unusual token activity, detecting volume spikes, and proposing high-confidence trade signals to human operators. All trade execution is client-side via MetaMask – SPECTER never holds or touches user funds. Every cycle is cryptographically logged and publicly auditable on GitHub.

// TABLE OF CONTENTS

1. Project Overview
2. Architecture
3. Agent Cycle – How It Works
4. Data Sources
5. Signal Detection & Spike Analysis
6. Trade Proposal System (Human-in-the-Loop)
7. Security Model
8. Client-Side Swap Execution
9. Positions & P&L; Tracking
10. Burn Mechanics
11. Staking System
12. Access Tiers (\$SPECTER)
13. Technical Stack
14. Smart Contracts
15. Transparency & Auditability
16. Limitations & Disclaimers

// 1. PROJECT OVERVIEW

SPECTER is a fully autonomous on-chain intelligence system built on Base Network. It operates without a centralized server – the agent runs inside GitHub Actions, a free CI/CD platform, triggered every 30 minutes by a cron schedule. Results are stored as JSON files committed directly to the public GitHub repository, making every cycle fully transparent and auditable by anyone.

The system consists of three main components: a **Watcher** that fetches live token data from Bankr and DexScreener APIs, a **Core Agent** powered by Claude AI that analyzes the data and makes decisions, and a **Web Dashboard** that displays intelligence in real-time and enables human-approved trade execution via MetaMask.

■ Core Principles

Principle	Description
Non-custodial	SPECTER never holds or controls user funds. All trades execute client-side via MetaMask
Transparent	Every agent cycle is committed to a public GitHub repo. Nothing is hidden.
Autonomous	Runs every 30 minutes without human intervention via GitHub Actions cron.
Human-in-loop	Trade proposals require explicit human approval before any swap executes.
Open source	Full codebase is publicly auditable at github.com/ariam2025/specter .

// 2. ARCHITECTURE

SPECTER uses a serverless architecture where GitHub Actions serves as the execution environment. There is no backend server, no database, and no centralized infrastructure that can be hacked or go offline. The system is entirely event-driven.

■ System Diagram

Layer	Component	Technology
Scheduler	GitHub Actions Cron	Every 30 min (UTC)
Data Layer	Watcher Agent	Node.js // Bankr API + DexScreener
Intelligence	Core Agent (Claude AI)	Anthropic claude-sonnet-4-6
Storage	GitHub Repository	JSON files committed to /data /memory /proofs
Frontend	Web Dashboard	Vanilla HTML/CSS/JS on GitHub Pages
Execution	MetaMask + ethers.js	Client-side only // Uniswap v3 SwapRouter02
Contracts	ERC-20 + Staking	Base Network (chainId 8453)

■ Data Flow

GitHub Actions trigger (cron / manual)

- > agents/watcher/index.js – fetch token data from Bankr + DexScreener
- > agents/core/run.js – Claude AI analyzes, writes, proposes
- > data/tokens.json – all tracked tokens
- > data/spikes.json – detected spike signals
- > memory/learnings.md – agent learnings across cycles
- > proofs/{date}/cycle-N.json – immutable cycle proof
- > GitHub Issues – trade proposals + signal flags
- > git commit + push – all results public on GitHub

Web Dashboard (GitHub Pages)

- > fetches raw.githubusercontent.com – reads JSON files
- > fetches GitHub Issues API – reads trade proposals
- > MetaMask (user browser) – executes approved trades

// 3. AGENT CYCLE – HOW IT WORKS

Each cycle follows a deterministic sequence. The agent wakes, gathers context, reasons about the data using Claude AI, takes actions via a tool-use loop, and commits proof before sleeping until the next cycle.

Step	Action	Output
1	Watcher fetches Bankr /token-launches	Raw token list with metadata
2	Enrich with DexScreener price/volume data	Spike ratios, liquidity, buy/sell counts
3	Write data/tokens.json + data/spikes.json	Public JSON files on GitHub
4	Core agent builds context from all data	Market summary + pending trades
5	Claude AI reasons via think() tool	Internal analysis before acting
6	Agent writes cycle journal	memory/cycles/{date}/{timestamp}.md
7	Agent updates learnings if new patterns	memory/learnings.md
8	Agent creates GitHub Issues for signals	Issues with label "signal"
9	Agent proposes trades for HIGH signals	Issues with label "trade-proposal"
10	Agent updates data/intelligence.md	Human-readable summary
11	Cycle proof committed to proofs/	Immutable audit trail
12	git push – everything public	Verifiable on GitHub

// 4. DATA SOURCES

Source	Endpoint	Data Provided
Bankr	api.bankr.bot/token-launches	New token launches, creator handles, tweet URLs
DexScreener	api.dexscreener.com/token-pairs/v1	Price, volume, liquidity, buy/sell counts
GitHub API	api.github.com/repos/{repo}/issues	Trade proposals, signal issues, comments
CoinGecko	api.coingecko.com/simple/price	ETH/USD price for P&L calculation

Bankr is the primary source for token discovery. Every token launched via the Bankr launchpad uses the Doppler AMM (Uniswap v4-based). DexScreener provides the on-chain trading data needed to calculate spike ratios and liquidity depth.

// 5. SIGNAL DETECTION & SPIKE ANALYSIS

SPECTER calculates a "spike ratio" for each token – the ratio of the last hour's volume compared to the average hourly volume over 24 hours. A spike ratio above 3x indicates unusual activity worth monitoring. A ratio above 5x triggers a GitHub Issue alert.

■ Spike Ratio Formula

$\text{spikeRatio} = \text{vol1h} / (\text{vol24h} / 24)$

Special case: $\text{spikeRatio} = 99$

→ Brand new token with 1h volume but no 24h baseline

→ Treated as MAXIMUM INTEREST signal

Spike Ratio	Classification	Agent Action
< 3x	Normal activity	Monitor only
3x - 5x	Elevated signal	Flag in spike list, monitor
5x - 8x	HIGH signal	Create GitHub Issue, consider proposal
≥ 8x	EXTREME signal	Create Issue + evaluate trade proposal
= 99	New launch (no baseline)	Immediate attention, high priority

■ Trade Proposal Confidence Criteria

Level	Criteria	Action
HIGH	$\text{spike} \geq 8x$ AND $\text{buys/sells} \geq 5:1$ AND liq locked/renounced AND proposal trade	Propose trade
MEDIUM	$\text{spike} \geq 5x$ AND $\text{buys/sells} \geq 4:1$ AND any positive signal	Propose trade
LOW	$\text{spike} \geq 3x$ but mixed signals	Signal issue only

// 6. TRADE PROPOSAL SYSTEM (HUMAN-IN-THE-LOOP)

SPECTER never executes trades automatically. When the agent detects a HIGH or MEDIUM confidence signal, it creates a GitHub Issue as a formal trade proposal containing full analysis, suggested size, risks, and the token contract address. The human operator reviews the proposal and either approves or rejects it directly in the GitHub Issue comments.

■ Approval Flow

1. Agent detects spike → evaluates confidence criteria
2. Agent calls `propose_trade()` → creates GitHub Issue with label "trade-proposal"
3. Issue contains: token CA, spike ratio, volume, analysis, risks, suggested size
4. Human reviews proposal on GitHub or Dashboard
- 5a. APPROVE: Comment "APPROVE \$" on the issue
- 5b. REJECT: Comment "REJECT" on the issue
6. Dashboard reads comments → shows APPROVE & SWAP button if approved
7. Human clicks APPROVE & SWAP → MetaMask opens for confirmation
8. Human signs transaction → swap executes from their wallet
9. Next agent cycle detects REJECTED issues → closes them automatically

■ The agent reads pending trade approvals each cycle via the GitHub Issues API. It scans comments for APPROVE/REJECT patterns and updates its context accordingly.

// 7. SECURITY MODEL

Security is the foundational design principle of SPECTER. Every architectural decision was made to ensure user funds are never at risk from the system itself.

Security Property	Implementation
Non-custodial	SPECTER has zero access to user wallets or private keys at any point.
No server-side keys	No API keys or credentials stored in the codebase or accessible to the agent.
Client-side execution	All swaps execute in the user's browser via MetaMask. Transactions are signed 1
Human approval gate	No trade can execute without explicit human review and MetaMask confirmation.
Open source	Full codebase public on GitHub. Anyone can audit every line of code.
Immutable audit trail	Every cycle committed to Git with timestamp. History cannot be altered.
No admin keys	Smart contracts will be ownership-renounced after deployment.
GitHub Secrets only	ANTHROPIC_API_KEY stored as encrypted GitHub Secret, never in code.

■ What SPECTER Cannot Do

X Access your wallet or private keys

// 8. CLIENT-SIDE SWAP EXECUTION

When a user approves a trade proposal on the dashboard, the swap executes entirely in their browser using ethers.js v6 and MetaMask. SPECTER's server (GitHub Actions) is not involved in any part of the transaction.

■ Swap Technical Details

Parameter	Value
Network	Base Network (chainId 8453)
DEX	Uniswap v3 (SwapRouter02)
Router address	0x2626664c2603336E57B271c5C0b26F421741e481
WETH (Base)	0x420006
Fee tiers tried	10000 (1%) → 3000 (0.3%) → 500 (0.05%) – auto-selects available pool
Slippage	No minimum output set – user is warned of high risk
Execution	exactInputSingle: WETH → Token (buy) or Token → WETH (sell)
Fallback	If no v3 pool found, dashboard shows DexScreener link for manual swap

■ Bankr-launched tokens use the Doppler AMM (Uniswap v4-based). If no Uniswap v3 pool exists, the dashboard automatically shows a DexScreener fallback link where the user can execute the swap manually through the native Doppler interface.

// 9. POSITIONS & P&L; TRACKING

After a successful swap, SPECTER records the position in the browser's localStorage. No position data is ever sent to a server. The dashboard refreshes prices every 60 seconds to provide live unrealized P&L.;

Field	Description
symbol	Token ticker symbol
tokenAddress	ERC-20 contract address on Base
amountEth	ETH spent in the swap
tokenAmount	Exact tokens received (from Transfer event log)
entryPriceUsd	Calculated entry price in USD
ethPriceAtEntry	ETH/USD price at time of entry (from CoinGecko)
entryTimestamp	Unix timestamp of the swap
txHash	On-chain transaction hash (verifiable on BaseScan)
status	"open" or "closed"
closePnlUsd	Realized P&L in USD (populated on sell)

// 10. BURN MECHANICS

Users can burn \$SPECTER tokens to request specific intelligence actions from the agent. Burned tokens are sent to the treasury address via an on-chain ERC-20 transfer. The burn triggers a GitHub Issue pre-filled with the request – SPECTER picks it up on the next cycle and processes it automatically.

Action	Cost	Description
Token Analysis	100 \$SPECTER	Deep analysis of any Base token: contract, socials, wallet concentr
24H Monitor	500 \$SPECTER	24-hour priority monitoring on a specific token. Every cycle checks
Smart Contract Audit	1,000 \$SPECTER	Full autonomous audit: rug vectors, honeypot indicators, mint funct

■ Burn Flow

1. User enters token contract address on Dashboard → clicks BURN & REQUEST
 2. MetaMask: approve \$SPECTER spend to treasury address
 3. MetaMask: transfer \$SPECTER to treasury (on-chain, verifiable)
 4. Dashboard opens GitHub Issues with pre-filled burn request (label: burn-request)
 5. Agent detects the issue on next cycle → executes the requested action
 6. Results posted as comment on the issue
- Burned tokens flow to the treasury. Treasury revenue is distributed to \$SPECTER stakers.

// 11. STAKING SYSTEM

Users who stake \$SPECTER tokens earn a share of platform revenue generated by burn fees. The staking contract follows the Synthetix rewards distribution model – a battle-tested pattern used across DeFi. Stakers earn proportionally to their share of the total staked pool.

Parameter	Value
Staking token	\$SPECTER (ERC-20 on Base)
Reward source	Treasury receives burn fees → distributes to stakers
Contract model	Synthetix StakingRewards pattern
Functions	stake(), withdraw(), getReward()
Read functions	balanceOf(address), earned(address), totalSupply()
Status	Contract deployment pending – addresses will be published on GitHub

// 12. ACCESS TIERS (\$SPECTER)

Dashboard features are gated by \$SPECTER token balance. The tier system is enforced client-side by reading the connected wallet's on-chain token balance via ethers.js. No server-side authentication is required.

Tier	Requirement	Access
// FREE	0 \$SPECTER	Public dashboard, delayed data, spike list
// SCOUT	1,000 \$SPECTER	Real-time signals, spike alerts
// HUNTER	10,000 \$SPECTER	Agent log access, memory viewer, raw cycle proofs
// PHANTOM	100,000 \$SPECTER	Raw proof access, API integration, priority signals

■ Tier verification happens in wallet.js. The contract reads balanceOf(address) directly from the \$SPECTER ERC-20 on Base. No off-chain database or authentication server involved.

// 13. TECHNICAL STACK

Component	Technology	Version / Details
Agent runtime	Node.js	v20+ (LTS)
AI model	Anthropic Claude	claude-sonnet-4-6
AI SDK	@anthropic-ai/sdk	^0.34.0
HTTP	Native fetch (Node)	Built-in (Node 20+)
Frontend	Vanilla HTML/CSS/JS	No framework – zero dependencies
Web3 library	ethers.js	v6.7.0 (CDN)
Hosting (web)	GitHub Pages	Free, CDN-backed
CI/CD	GitHub Actions	Cron every 30 min
Data storage	Git repository	JSON files, Markdown
Package manager	npm	ES modules ("type": "module")
Target chain	Base Network	chainId 8453 (Coinbase L2)
DEX	Uniswap v3	SwapRouter02 on Base

// 14. SMART CONTRACTS

Three smart contracts are part of the SPECTER ecosystem, all deployed on Base Network. Addresses will be published in the GitHub repository and this document updated upon deployment.

Contract	Address	Purpose
\$SPECTER ERC-20	TBD – pending deployment	Utility token for tier access, burns, staking
Treasury	TBD – pending deployment	Receives burn fees, distributes to stakers
StakingRewards	TBD – pending deployment	Synthetix-model staking, reward distribution

■ Contract Security Measures

- Ownership will be renounced after deployment and verification
- No mint function accessible post-launch
- No proxy / upgrade pattern – immutable contracts
- Source code will be verified on Basescan
- All contract addresses published in GitHub repository

// 15. TRANSPARENCY & AUDITABILITY

Every action SPECTER takes is publicly logged on GitHub. This creates an immutable, timestamped audit trail that anyone can verify independently.

Artifact	Location	Contents
Cycle proofs	proofs/{date}/cycle-N.json	Full agent step-by-step reasoning and tool calls
Token data	data/tokens.json	All tracked tokens with prices and volumes
Spike signals	data/spikes.json	Detected anomalies with ratios and metadata
Agent memory	memory/learnings.md	Patterns learned across all cycles
Agent identity	memory/self.md	Agent's self-description and operating principles
Current focus	memory/focus.md	Active objectives and priorities
Cycle journals	memory/cycles/{date}/	Per-cycle narrative written by the agent
Intelligence log	data/intelligence.md	Human-readable summary of latest findings
Trade proposals	GitHub Issues	Public issues with full analysis and approval trail

The GitHub repository acts as a decentralized, tamper-evident ledger of all agent activity. Each commit includes a timestamp, cycle number, and cryptographic hash linking to the previous state. This means even the project operators cannot retroactively alter agent history without it being detectable.

// 16. LIMITATIONS & DISCLAIMERS

1. SPECTER IS NOT FINANCIAL ADVICE. All signals, proposals, and analyses are for informational purposes only. Never invest more than you can afford to lose.
2. NEW TOKEN TRADING IS EXTREMELY HIGH RISK. Bankr-launched tokens are brand new with no track record. Most will go to zero. Rug pulls and honeypots are common.
3. NO SLIPPAGE PROTECTION. Swaps execute with `amountOutMinimum = 0`. You may receive significantly less tokens than expected due to price impact.
4. DOPPLER (v4) TOKENS. Some Bankr tokens use Doppler AMM which is Uniswap v4-based and incompatible with the v3 SwapRouter. These will show a DexScreener fallback link.
5. AI ERRORS. Claude AI can make mistakes in analysis. Always do your own research before executing any trade.
6. GITHUB ACTIONS DELAYS. The agent runs every 30 minutes. In fast-moving markets, signals may be stale by the time you review them.
7. GITHUB RATE LIMITS. The dashboard reads from the GitHub API which has rate limits. Heavy usage may result in temporary throttling.
8. BROWSER STORAGE. Positions are stored in `localStorage`. Clearing browser data will delete position history.
9. BETA SOFTWARE. SPECTER v1.0 is beta software. Use at your own risk. The operators are not liable for any financial losses.
10. NOT AUDITED. Smart contracts have not been formally audited at launch. A third-party audit is planned post-deployment.

SPECTER // BASE NETWORK INTELLIGENCE

AUTONOMOUS AGENT // ALWAYS SCANNING // NEVER SLEEPING

github.com/ariam2025/specter